



Arquitectura y motor de decisión.

Señales SOL/USDT con disciplina sistemática.

Presentación técnica · documento interno

SECCION 01

Resumen del sistema

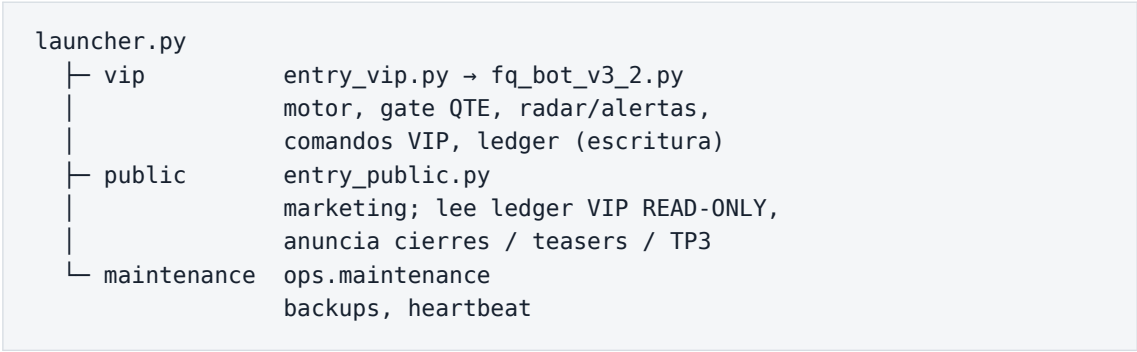
FQ es un bot de **trading + pagos en vivo** sobre SOL/USDT en producción (Railway). Vigila el mercado en tiempo real, evalúa setups contra un pipeline de varias fases y solo dispara cuando superan el gate de convicción. Cada señal se sigue hasta su cierre y el outcome alimenta la memoria estadística.

- ▶ **~18,000 líneas** en total; el monolito `fq_bot_v3_2.py` concentra ~4,900.
- ▶ **Dos caras:** bot VIP (motor + señales) y bot público (marketing, read-only).
- ▶ **Runtime ligero:** numpy + pandas + LLM on-demand. Sin libs cuánticas ni ML pesado.

SECCION 02

Topología de procesos

`launcher.py` arranca 3 procesos hijos en el **mismo** servicio Railway (no permite volúmenes compartidos entre servicios), sobre el mismo volumen:



Comunicación **VIP → Público:** **solo** vía el SQLite del ledger (RO). Cero acoplamiento de código. Si un hijo muere, el launcher sale con `rc != 0` y Railway reinicia el contenedor.

SECCION 03

Stack

Capa	Tecnología
Lenguaje	Python 3.11 (ASCII-only en módulos de motor)
Datos de mercado	CCXT (fetch_ohlcv) + pandas / pandas-ta
Cómputo	numpy / pandas
Persistencia	SQLite (ledger append-only; RO para el público)
Mensajería	Telegram Bot API (long polling)
LLM	Anthropic — Sonnet (lecturas) + Opus (revisión / audit)
Web / health	Flask
Deploy	Railway (railway.toml, Procfile)

SECCION 04

Pipeline de decisión

`fusion_engine.evaluate_signal()` es la **única** función pública del motor. Evalúa en fases:

- **Fase A — Sesgo estructural + PD.** Dirección y zona Premium/Discount.
- **Fase B — Liquidez.** Sweeps, pools no barridos, calidad de reacción.
- **Fase C — Confluencia ICT.** Requiere $\text{CONFLUENCE_MIN} \geq 3$; +4% por concepto (cap +16%).
- **Fase D — Timing + CRT + memoria.** Bucket de outcome → kappa_evo ($\pm 15\%$ sobre P_master).
- **Fase E — Sync modulator.** Tiempo emergente tau; $\text{sync_score} < 0.30$ → veto absoluto.
- **Volume quality** (v5.2) — modulador final sobre P_master .

Gate de convicción: $\text{P_master} \geq \phi^2$ (≈ 2.618). Para fire/broadcast, CONSTRAINTS §6 exige $\text{P_master} \geq 7/10$ (~ 2.965). El gate Theta(D) es sagrado: ningún modulador lo toca.

Capa ML (v4.3, additiva y advisory): ensemble de 5 scorers (volume, structure, liquidity, concept_stack, history) + atribución Shapley. Enriquece la decisión cuando P_master ya pasó el umbral; no sustituye al motor.

SECCION 05

Mapa de módulos

Núcleo de decisión (puro, testeable)	<code>fusion_engine · quantum_timelines</code> <code>battle_planner · signal_scorer</code> <code>ict_smc · market_context · regime_detector</code> <code>volume_quality · killzones_pd</code> <code>emergent_time · entropy_cognition</code> <code>fq_radar · fq_market_data</code> (extraídos v5.3)
Presentación (blindada x tests)	<code>branding · vip_format · public_format</code> <code>field_reports · legal</code>
Bot público	<code>public_outcome_announcer · public_scheduler</code> <code>public_handlers · public_content_generator</code>
Pagos / suscripción	<code>vip_system · payments</code>
Monolito (a adelgazar)	<code>fq_bot_v3_2</code> (loop, broadcast, routing, hooks)

SECCION 06

Motor QTE (Quantum Timelines Engine)

Motor probabilístico **cuántico-inspirado** (sin qiskit ni dimod). Simula N futuros (Monte Carlo paths) bajo restricciones estructurales y mide P(TP_i), P(SL), valor esperado en R y régimen dominante.

Concepto cuántico	Mapeo clásico
Superposición	N paths Monte Carlo coexistiendo
Función de onda	Distribución empírica de paths
Hamiltoniano	bias: drift + vol + magnetic_pull
QAOA	grid search sobre (SL, TP1, TP2, TP3) → max EV
Colapso / medición	régimen modal del ensemble
Decoherencia	divergencia de paths = incertidumbre

- ▶ **300-2000 paths**, horizonte 96 velas de 15m (= 24h).
- ▶ Performance objetivo: **< 1.5s con 500 paths** en Railway.
- ▶ Módulo **inerte**: solo computa, no envía ni toca DB.

SECCION 07

Memoria y autoevolución

entropy_cognition — el bot no recuerda outcomes individuales, los **destila** en distribuciones:

- ▶ **SignalLedger** — SQLite append-only, backup a Telegram.
- ▶ **OutcomeTracker** — monitorea señales abiertas hasta TP/SL/timeout.
- ▶ **EntropyEngine** — Shannon H sobre buckets (sesión×tier×dirección×curvatura) + KL drift.
- ▶ **KappaEvo** — modulador suave ±15% sobre P_master. Nunca toca Theta(D).
- ▶ **SelfAudit** — cada 25 cerradas, Opus audita y propone ajustes (no autoaplicados).

emergent_time — postulado $\tau = \phi_{clock} \cdot \phi_{memory} \cdot \phi_{horizon} \cdot \phi_{refractory}$.
Módulo puro. battle_planner combina campo ICT + paths QTE y emite: EJECUTAR_AHORA / ACUMULAR_EN_ZONA / ESPERAR_GATILLO / STAND_DOWN.

SECCION 08

Co-pilot LLM

`claude_integration.py` — LLM táctico on-demand:

- ▶ **Sonnet** — lecturas VIP y análisis a pedido.
- ▶ **Opus** — revisión de señales auto-disparadas de alta convicción y self-audit.

Recibe payloads ricos (precio, QTE, niveles, eventos, walls, derivados). En superficies de cliente el output es **cualitativo** (probabilidad alta / edge claro), nunca fórmulas ni jerga del motor.

SECCION 09

Migración y roadmap

Estrategia **strangler fig**: extraer piezas del monolito a módulos puros/testeables, por etapas verificadas y reversibles. Regla de oro: extraer → el monolito importa/reexporta → pytest verde antes de commitear.

- ▶ `[hecho] fq_radar.py` — decisión pura del radar.
- ▶ `[hecho] fq_market_data.py` — `fetch_ohlc` + `add_indicators` mockeables.
- ▶ `[ ] fq_broadcast.py` — envío Telegram desacoplado de globals.
- ▶ `[ ] fq_commands/` — router de comandos (hoy un gran `if/elif`).
- ▶ `[ ] fq_signal_loop.py` — loop principal + hooks como orquestador.
- ▶ `[ ]` paquete `fq/` — agrupar piezas; monolito como shim de arranque.

SECCION 10

Garantías y pruebas

- ▶ `pytest` como suite principal + self-tests embebidos (`emergent_time`, `quantum_timelines`, `battle_planner`).
- ▶ `tests/test_client_surfaces.py` blindo las superficies de cliente: caza filtraciones de versión, modelo, framework o fórmula.
- ▶ Toda constante de tuning es override-able por `env` (defaults en el módulo).
- ▶ Módulos de motor en ASCII-only; sin side effects al importar.

FQ · Documento técnico interno. No distribuir como material de cliente.